LEÇON

01

Démarrer

Premier programme, outillage et style GNAT

PHASE 1 — FONDAMENTAUX

OBJECTIFS DE LA LEÇON

01 Comprendre pourquoi Ada est le langage des systèmes critiques

02 Installer Alire, GNAT et GPRbuild sans droits administrateur

03 Créer, compiler et lancer un projet avec alr

04 Lire la structure d'un projet : alire.toml, .gpr, src/

05 Écrire un programme : with, use, procedure, begin, end

06 Afficher du texte avec Ada.Text_IO

07 Appliquer le style GNAT et obtenir un build silencieux

08 Organiser et versionner le dépôt learning-ada

HISTOIRE ET USAGES

1 Pourquoi Ada ?

À la fin des années 1970, le Département de la Défense américain (DoD) utilise **plus de 450 langages** dans ses systèmes. La maintenance coûte une fortune et les bugs sont fréquents. Le DoD lance un concours pour créer un langage unique, fiable et maintenable. Il est remporté par une équipe **française**, celle de **Jean Ichbiah** chez CII Honeywell Bull. Le langage est baptisé Ada, en hommage à Ada Lovelace ; sa première norme porte le numéro MIL-STD-**1815**, l'année de naissance de la mathématicienne.

Version	Apports majeurs
Ada 83	Typage fort, paquetages, tâches : la concurrence est intégrée au langage
Ada 95	Programmation objet, objets protégés
Ada 2005	Interfaces, conteneurs standards
Ada 2012	Contrats : Pre, Post, invariants de type
Ada 2022	Parallélisme, nombreuses améliorations d'expressivité

On trouve Ada partout où une erreur logicielle peut coûter des vies ou des milliards : avionique civile et militaire, lanceurs et satellites, signalisation ferroviaire, systèmes d'armes et radars. Ces systèmes vivent souvent **30 ou 40 ans** : il faut des ingénieurs pour les maintenir et les faire évoluer.

La philosophie : détecter les erreurs le plus tôt possible

	C	Python	Ada
Typage	Statique mais faible, conversions implicites	Dynamique	Statique et **très fort**, aucune conversion implicite
Débordement de tableau	Comportement indéfini	Exception à l'exécution	Exception, souvent détecté dès la compilation
Débordement d'entier	Indéfini ou silencieux	Impossible (entiers illimités)	`Constraint_Error`
Priorité	Performance, liberté	Rapidité d'écriture	**Lisibilité et fiabilité**

> **COMPARAISON C / PYTHON**
>
> Un commentaire peut mentir, un type non. Si un commentaire dit « altitude en mètres » et qu'on y met des pieds, rien ne bronche. Si le **type** s'appelle `Altitude_M`, le compilateur vérifie chaque utilisation, sans jamais se fatiguer. En Ada, une grande partie de la documentation est **vérifiée par la machine**.

CÔTÉ DÉFENSE

Ada est conçu pour être **lu** bien plus souvent qu'écrit : un code de défense est relu, audité et certifié, parfois des décennies plus tard. La syntaxe verbeuse (`begin`/`end`, mots-clés plutôt que symboles) est un choix délibéré.

Ariane 501 (1996) : une conversion d'un flottant 64 bits vers un entier 16 bits a débordé. Ada détectait l'erreur, mais la vérification avait été désactivée sur cette conversion et l'exception n'était pas traitée. Leçon de l'industrie : une vérification désactivée sans justification est une bombe. SPARK permet de **prouver** qu'un tel débordement est impossible (leçons 19 et 20).

À RETENIR

Le coût d'un bug grandit avec le temps : quelques secondes à la compilation, des heures en test, des jours en intégration… et une fusée en vol. Ce que le langage refuse tôt est un investissement, pas un frein.

2

SANS DROITS ADMINISTRATEUR

Installer l'environnement

Alire (commande `alr`) est le gestionnaire de projets et de paquets d'Ada. Il télécharge le compilateur **GNAT** et l'outil de build **GPRbuild**, crée les projets et gère les dépendances. Tout s'installe dans le dossier de l'utilisateur.

Python	C	Ada
`pip` + `venv`	`make` / CMake + compilateur installé à la main	`alr`

Les quatre étapes

- **Télécharger** la version zip : `alr-X.Y.Z-bin-x86_64-windows.zip` sur github.com/alire-project/alire/releases, et l'extraire dans un dossier sans espaces ni accents, par exemple `C:\Users\<nom>\tools\alire`.
- **Ajouter au PATH du compte** le dossier `...\alire\bin` via « Modifier les variables d'environnement pour votre compte ». Fermer ensuite tous les terminaux et VS Code.
- **Choisir la chaîne de compilation** : `alr toolchain --select`, accepter l'installation de MSYS2, puis choisir `gnat_native` et `gprbuild`.
- **Installer l'extension** « Ada & SPARK » éditée par **AdaCore** dans VS Code.

```
VÉRIFIER L'INSTALLATION
alr version          # version d'Alire et dossiers utilisés
alr toolchain        # outils installés : gnat_native, gprbuild
```

PIÈGE

L'installateur `.exe` d'Alire vise `C:\Program Files` et demande les droits admin. Sur un poste d'entreprise, prendre la version **zip** et modifier le PATH **du compte**, jamais celui du système.

ASTUCE

Si un antivirus ou un proxy bloque les téléchargements d'Alire, noter le message d'erreur exact : c'est le problème le plus fréquent sur un poste sans droits admin.

3

ALR INIT, ALR BUILD, ALR RUN

Premier projet

```
CRÉER, COMPILER, LANCER
mkdir learning-ada
cd learning-ada
alr init --bin lesson_01      # --bin : exécutable ; --lib : bibliothèque
cd lesson_01
alr build                      # compiler
alr run                        # compiler si besoin, puis exécuter
code .                         # ouvrir le projet dans VS Code
```

Ce que fait vraiment « alr build »

Le C a deux étapes : compilation puis édition de liens. Ada en ajoute une troisième au milieu, le **binder**.



- Il **vérifie la cohérence** du programme entier : si une unité a changé sans que celles qui en dépendent soient recompilées, il refuse de continuer. En C, un `.h` modifié peut produire un exécutable incohérent sans que le linker ne dise rien.
- Il calcule l'**ordre d'élaboration** : les paquetages peuvent exécuter du code d'initialisation (comme le code au niveau module en Python, exécuté à l'`import`). Il génère `b__lesson_01.adb`, le vrai point d'entrée.

La structure d'un projet Alire

Élément	Rôle	Git
`src/`	**Le code source.** Le seul dossier où l'on écrit.	Oui
`alire.toml`	Manifeste : nom, version, auteurs, licence, options de build. Équivalent du `pyproject.toml`.	Oui
`lesson_01.gpr`	Projet GPRbuild : sources, programmes principaux, options du compilateur. Équivalent du `Makefile`.	Oui
`config/`	Généré par Alire à chaque build. **Ne jamais le modifier.**	Non
`obj/`, `bin/`, `alire/`	Fichiers objets, exécutables, cuisine interne d'Alire.	Non
`share/`	Ressources éventuelles (données…).	Si non vide

```
LESSON_01.GPR (EXTRAIT) — UNE SYNTAXE PROCHE D'ADA
with "config/lesson_01_config.gpr";
project Lesson_01 is
   for Source_Dirs use ("src/", "config/");
   for Object_Dir use "obj/" & Lesson_01_Config.Build_Profile;
   for Exec_Dir use "bin";
   for Main use ("lesson_01.adb", "ex01_banner.adb");  --  un main par exercice

   package Compiler is
      for Default_Switches ("Ada") use Lesson_01_Config.Ada_Compiler_Switches;
   end Compiler;
end Lesson_01;
```

À RETENIR

Pour changer une option de compilation, on modifie `alire.toml` ou le `.gpr`, jamais les fichiers de `config/`. Et on ne versionne jamais ce que la machine peut régénérer.

4

WITH, USE, PROCEDURE, BEGIN, END

Anatomie d'un programme

```
HELLO.ADB
--  Prints a greeting and an empty line.
with Ada.Text_IO; use Ada.Text_IO;     --  clause de contexte

procedure Hello is                      --  le programme principal
   --  Zone déclarative : uniquement des déclarations
begin
   --  Zone des instructions
   Put_Line ("Hello, ""Ada""!");       --  guillemet doublé = un guillemet
   New_Line;
end Hello;                              --  le nom est vérifié
```

Élément	Ce qu'il faut retenir
`procedure ... is`	Le programme principal est une procédure sans paramètre. Pas de `main` imposé, pas de `return 0;`.
`is` … `begin`	Zone déclarative. On ne déclare jamais au milieu des instructions, comme en C89.
`null;`	Instruction « ne rien faire », équivalent du `pass` de Python. Un bloc vide est interdit.
`end Hello;`	Nom facultatif, mais **s'il est écrit, il doit être exact** : un nom faux est une erreur de compilation.
`'A'` / `"Ada"`	Apostrophes : **un** caractère. Guillemets : une chaîne. `'NOMINAL'` est une erreur.

with et use

	with	use
Rôle	Rend le paquetage **accessible**	Rend ses noms **visibles sans préfixe**
Obligatoire ?	Oui, pour utiliser le paquetage	Non, c'est un confort
Python	`import os`	`from os import *`
Seul ?	Oui	Non : `"Ada" is undefined`

COMPARAISON C / PYTHON

`with` n'est **pas** un `#include` : rien n'est copié dans ton fichier. Le compilateur lit la spécification déjà vérifiée du paquetage, et le binder contrôle ensuite la cohérence de l'ensemble. Pas de gardes `#ifndef`, pas de macros.

CÔTÉ DÉFENSE

Beaucoup de normes de codage limitent `use` sur les paquetages applicatifs : avec le nom complet, le relecteur sait immédiatement d'où vient chaque appel.

Afficher du texte : Ada.Text_IO

Procédure	Effet	C	Python
`Put ("x");`	Affiche sans retour à la ligne	`printf("x");`	`print("x", end="")`
`Put_Line ("x");`	Affiche puis va à la ligne	`puts("x");`	`print("x")`
`New_Line;`	Retour à la ligne (paramètre `Spacing`, défaut 1)	`putchar('\n');`	`print()`
`Set_Col (15);`	Va en colonne 15 ; si elle est dépassée, passe d'abord à la ligne	—	—

PIÈGE

Sans argument, on écrit `New_Line;` : des parenthèses vides `()` sont interdites en Ada. Et `Put` n'affiche que du texte : pas de `Put (42);` direct, le typage fort impose une conversion (leçon 2).

ASTUCE

La meilleure documentation d'Ada est souvent le code source de la bibliothèque standard. Dans VS Code, **F12** sur `New_Line` ouvre `a-textio.ads` à la bonne ligne : on y lit le nom, le type et la valeur par défaut de chaque paramètre.

LE STYLE GNAT

5 Commentaires et conventions

```
COMMENTAIRES
--  Commentaire seul sur sa ligne : DEUX espaces après --
Put_Line ("Hi");  -- en fin de ligne, un seul espace suffit
--  Pas de commentaire de bloc : chaque ligne porte son --
Put_Line ("-- ceci n'est pas un commentaire --");  --  c'est une chaîne
```

- Ada **ne distingue pas** majuscules et minuscules, mais le style GNAT exige la **cohérence** : mots-clés en minuscules (`-gnatyk`), et chaque nom écrit exactement comme à sa déclaration (`-gnatyr`).
- Indentation de **3 espaces** (`-gnaty3`), un espace avant la parenthèse d'un appel et après chaque virgule (`-gnatyt`).
- Le fichier porte le **nom de l'unité en minuscules** : `Ex01_Banner` vit dans `ex01_banner.adb`.

	C	Python	Ada
Variables, sous-programmes	`max_speed`	`max_speed`	`Max_Speed`
Constantes	`MAX_SPEED`	`MAX_SPEED`	`Max_Speed`
Types	`speed_t`	`MaxSpeed`	`Speed` ou `Speed_Type`
Acronymes	—	—	`Text_IO` : en majuscules

PIÈGE

Ada a environ 70 mots réservés, dont certains que l'on utilise souvent comme noms en Python : `type`, `range`, `body`, `access`, `record`, `delta`, `digits`. Impossible d'appeler une variable `Range`. Le compilateur impose aussi : pas de _ au début ni à la fin, jamais deux _ consécutifs.

À RETENIR

Un bon commentaire dit **ce que fait le programme** ou **pourquoi** ; le **quoi** se lit dans les noms. Le compilateur valide la forme d'un commentaire, jamais sa vérité : un commentaire faux est pire qu'aucun commentaire.

Exemples relevés en revue de code

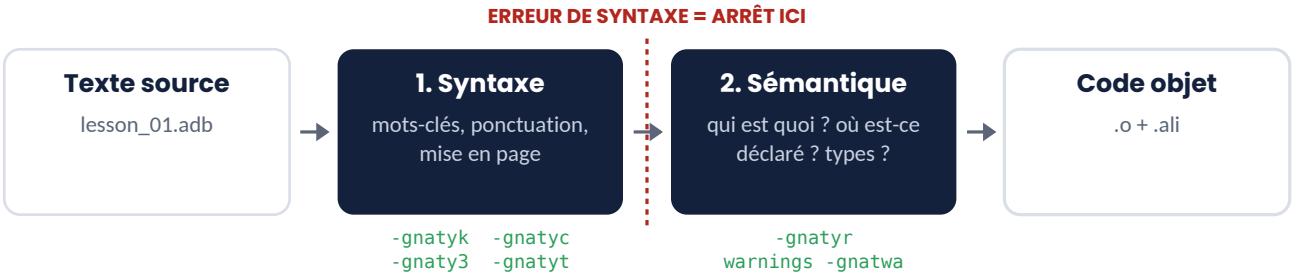
Commentaire	Problème
`-- this is the body` placé entre `is` et `begin`	Faux : c'est la zone déclarative, le corps commence après `begin`.
`-- Fixed exercise`	Ne dit rien de ce que fait le programme.
`-- Practice in debugging programs`	Décrit l'exercice, pas le programme.
`-- ... via the basic function`	`Put_Line` est une **procédure**, pas une fonction (leçon 5).

6

LA RÈGLE D'OR : ZÉRO MESSAGE

Warnings et style GNAT

Le compilateur travaille en **phases**. Une erreur de syntaxe arrête tout **avant** l'analyse sémantique : certains messages n'apparaissent donc qu'après avoir corrigé les premiers.



Famille	Options	Rôle
Débogage	`-Og -g`	Optimisation légère, informations de débogage
Warnings	`-gnatwa`	Tous les warnings utiles : variable inutilisée, code mort…
Validité	`-gnatVa`	Vérifie la validité des valeurs à l'exécution
Style	`-gnaty...`	Indentation, casse, commentaires, espaces, longueur de ligne
Bloquant	`-gnatwe`	Warnings et style deviennent des **erreurs**

Par défaut, les messages de style ne bloquent pas : le build affiche **Success** malgré eux. On rend la règle d'or automatique dans `alire.toml` :

```
ALIRE.TOML — DANS CHAQUE PROJET DU PARCOURS
[build-switches]
"*".compile_checks = "Errors"     # ajoute -gnatwe, pour tous les profils
```

```
FORMAT : FICHIER:LIGNE:COLONNE: GRAVITÉ: MESSAGE
show_info.adb:3:01: (style) reserved words must be all lower case [-gnatyk]
show_info.adb:6:04: (style) bad casing of "Put_Line" declared at a-textio.ads:565
lesson_01.adb:1:05: error: "Ada" is undefined
```

PIÈGE

Les warnings ne s'affichent qu'au moment où un fichier est **compilé**. Un build qui répond « up to date » sans message ne prouve rien. Pour vérifier : `alr build -- -f` (tout ce qui suit `--` va à GPRbuild, `-f` force la recompilation).

ASTUCE

Face à une cascade d'erreurs, corriger **la première** puis recompiler : les suivantes en sont souvent la conséquence. Et lire les suggestions de GNAT, qui propose souvent la correction exacte. En cas de doute sur une règle, **le compilateur est l'arbitre**.

CÔTÉ DÉFENSE

Dans un projet certifié, une norme de codage est imposée et vérifiée automatiquement : un fichier qui ne la respecte pas ne passe pas la revue. Prendre l'habitude du « zéro message » dès maintenant, c'est le réflexe attendu en entreprise.

7

GIT ET VITRINE GITHUB

Le dépôt learning-ada

UN MONODÉPÔT

```
learning-ada/
├── README.md        vitrine en anglais, une case cochée par leçon
├── LICENSE          MIT
└── lesson_01/       un projet Alire indépendant par leçon
    ├── alire.toml
    ├── lesson_01.gpr
    ├── .gitignore   généré par Alire : bin/ obj/ alire/ config/
    ├── README.md    cours, exercices, corrections séparées
    ├── lecon_01_premier_programme.pdf
    └── src/         un fichier par exercice
```

Bonne pratique	Pourquoi
Messages `lesson_NN: exercise N - sujet`	Un historique homogène se lit d'un coup d'œil : c'est la première chose qu'un recruteur voit.
Un commit, un sujet	Ajouter uniquement les fichiers concernés : `git status` puis `git add src\fichier.adb` plutôt que `git add .`.
`git commit --amend`	Corrige le dernier commit **avant** le push. Jamais après : on ne réécrit pas un historique public.
`git log --oneline -3`	Vérifier les messages avant de pousser.
Aucun binaire versionné	`bin/`, `obj/` et les `.exe` n'ont rien à faire dans une vitrine.

PIÈGE

Sous Windows PowerShell 5, la redirection `>` et `Out-File` écrivent en **UTF-16** : diffs illisibles et rendu GitHub peu fiable. Créer les fichiers texte dans VS Code (UTF-8), et activer « Insert Final Newline » pour que chaque fichier se termine par un retour à la ligne.

Fiche mémo — Leçon 1

COMMANDES

`alr init --bin nom` créer un exécutable
`alr build` compiler
`alr build -- -f` tout recompiler
`alr run` lancer le main d'`alire.toml`
`.\bin\ex01.exe` lancer un exercice
`alr toolchain` voir les outils

STRUCTURE

`with P; use P;` contexte
`procedure X is` / déclarations
`begin` / instructions
`end X;` nom vérifié
`null;` ne rien faire

ADA.TEXT_IO

`Put ("...")` sans retour à la ligne
`Put_Line ("...")` avec retour
`New_Line (Spacing => 3)` lignes vides
`Set_Col (15)` aller en colonne 15
`"Say ""hi"""` guillemets doublés

GIT

`git status` vérifier avant commit
`git add src\fichier.adb` un sujet
`git commit -m "lesson_01: ..."`
`git commit --amend` avant push seulement

STYLE GNAT

Mots-clés en minuscules, noms en `Mixed_Case`
Casse identique à la déclaration
Indentation de 3 espaces
`Put ("x")` : espace avant `(`
`--  texte` : deux espaces
Fichier = unité en minuscules

BUILD

Compile → **Bind** → Link
`"*".compile_checks = "Errors"`
Corriger la 1re erreur, recompiler
Syntaxe d'abord, sémantique ensuite
« up to date » ne prouve rien

PIÈGES

`use` sans `with` : erreur
`'NOMINAL'` : un seul caractère permis
`New_Line()` : parenthèses vides interdites
Mots réservés : `type`, `range`…
PowerShell `>` écrit en UTF-16

VS CODE

F12 : aller à la définition
Ctrl+: commenter / décommenter
Extension « Ada & SPARK » (AdaCore)
Fichiers en UTF-8, « Insert Final Newline »

Exercices

Un fichier par exercice dans `src/`, déclaré dans le `.gpr`. Build silencieux avec `alr build -- -f`, puis un commit par exercice. Les corrections sont dans le README de la leçon.

Exercice 1 — Bannière

Fichier `ex01_banner.adb`. Sans clause `use`, avec `Put`, `Put_Line` et `New_Line` au moins une fois chacune. La dernière ligne est construite en deux appels.

```
SORTIE ATTENDUE
+----------------------------+
|        LEARNING ADA        |
|  Lesson 01: first program  |
+----------------------------+

Author: Thomas Fuzellier
```

Exercice 2 — Briefing de mission

Fichier `ex02_mission_brief.adb`. Avec `use`, un commentaire par section. La ligne `Operation` en exactement trois appels ; les lignes vides uniquement avec `New_Line`. Avant de compiler, prédire si le `--` de la dernière ligne est un commentaire.

```
SORTIE ATTENDUE
============================
       MISSION BRIEFING
============================

Operation: "Silent Falcon"
Status:    READY
Priority:  HIGH

Orders:
  1. Compile without warnings.
  2. Review every line.
  3. Report to command.

-- End of briefing --
```

Exercice 3 — Code à réparer

Fichier `ex03_fix_me.adb`. Copier le code tel quel, puis corriger **une erreur à la fois** en lisant le compilateur. Noter pour chaque problème s'il s'agit de compilation ou de style. Un problème est invisible pour le compilateur.

CODE FOURNI

```
with Ada.Text_Io;
Use Ada.Text_IO;

procedure Ex03_Fix_Me is
   -- Fixed exercise
begin
  Put_Line ("Radar system online")
   put_line ("Tracking 3 targets");
   Put ("Status: ")
   Put_Line ('NOMINAL');
   New_Line();
end Ex03_FixMe;
```

Exercice 4 — Rapport de cibles

Fichier `ex04_target_report.adb`. Les trois lignes vides viennent d'**un seul** appel ; aucune chaîne ne contient d'espaces d'alignement, la colonne `Status` commence en colonne 15. Trouver les procédures dans `a-textio.ads` (F12 dans VS Code).

SORTIE ATTENDUE

```
TARGET REPORT



Name           Status
Alpha          Tracked
Bravo          Lost
Charlie        Tracked
```

Mini-quiz

Une seule bonne réponse par question. Réponses en bas de page.

1. Quel est le rôle du binder ?

a) Optimiser le code machine
b) Vérifier la cohérence du programme et calculer l'ordre d'élaboration
c) Traduire Ada en C
d) Appliquer le style -gnaty

2. Que se passe-t-il avec `use Ada.Text_IO;` sans `with` ?

a) Ça compile, use implique with
b) Ça compile avec un warning
c) Erreur : use s'applique à un paquetage déjà rendu visible par with
d) Erreur à l'édition de liens

3. Comment afficher `Operation: "Falcon"` ?

a) `Put_Line ("Operation: \"Falcon\"");`
b) `Put_Line ('Operation: "Falcon"');`
c) `Put_Line ("Operation: ""Falcon""");`
d) C'est impossible en Ada

4. Lequel de ces commentaires, seul sur sa ligne, est refusé par `-gnatyc` ?

a) `--  Compute the range.`
b) `-- Compute the range.`
c) `--`
d) `----------------`

5. Effet de `"*".compile_checks = "Errors"` ?

a) Désactive les warnings
b) Warnings et style bloquent le build (-gnatwe)
c) Active les vérifications à l'exécution
d) Ne concerne que le profil release

6. Le build affiche « up to date » sans message. Conclusion ?

a) Le code est sans warning
b) Rien : il faut `alr build -- -f` pour vérifier
c) Le binder a été sauté
d) Le style est désactivé

7. `procedure Lesson_01` se termine par `end Lesson_1;`. Résultat ?

a) Rien, le nom est décoratif
b) Un message de style -gnatye
c) Une erreur de compilation
d) Une erreur à l'exécution

8. Curseur en colonne 20, appel de `Set_Col (15)`. Que se passe-t-il ?

a) `Constraint_Error`
b) Rien, l'appel est ignoré
c) Le curseur recule et écrase le texte
d) Passage à la ligne, puis colonne 15

Réponses : 1 b · 2 c · 3 c · 4 b · 5 b · 6 b · 7 c · 8 d